

APPENDIX A

CODING

1. Logistic Regression

```
# This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input
directory

import os

for dirname, _, filenames in os.walk('/kaggle/input')
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that gets preserved as output
when you create a version using "Save & Run All"

# You can also write temporary files to /kaggle/temp/, but they won't be saved outside of the current
session

# upgrade sklearn

!pip install scikit-learn --upgrade
!pip install mlflow
!pip install dagshub
```

```

from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.ensemble import VotingClassifier, StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
import lightgbm as lgb
import numpy as np
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from sklearn.base import BaseEstimator, ClassifierMixin
from sklearn.metrics import accuracy_score
import numpy as np
import pandas as pd
import nltk
import re
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer, PorterStemmer
import mlflow
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.naive_bayes import MultinomialNB

```

```

from sklearn.model_selection import GridSearchCV
from sklearn.preprocessing import LabelEncoder
import json
import joblib
import os
import yaml
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from sklearn.model_selection import cross_val_score
nltk.download('all')
nltk.download('wordnet')
nltk.download('stopwords')
from nltk.stem import WordNetLemmatizer, PorterStemmer
import sklearn
sklearn.__version__
# load the data

DATA_PATH = "/kaggle/input/twitter-and-reddit-sentimental-analysis-dataset/Reddit_Data.csv"
df = pd.read_csv(DATA_PATH)

df.head()
import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

from google.colab import files
uploaded = files.upload()
import pandas as pd

```

```

df = pd.read_csv("Reddit_Data.csv") # Uses the uploaded file
df.head()
df.shape
df.rename({"clean_comment": "comment",
          "category": "sentiment"}, axis=1, inplace=True)
(
    df.isna().sum()
)
(
    df.loc[
        df['comment'].isna()
    ]
)
(
    df.loc[
        df['comment'].isna(), "sentiment"
    ]
    .value_counts()
)
# remove missing values
print("Rows in data before removing missing values ", df.shape[0])

df = df.dropna()

print("Rows in data after removing missing values ", df.shape[0])
from sklearn.utils import resample

# Optional but safe: drop duplicate comments
df = df.drop_duplicates(subset=['comment'])
print("Rows in data after removing duplicate comments: ", df.shape[0])

```

```

# Check original distribution
print("Original class distribution:")
print(df['sentiment'].value_counts())

# Separate classes
df_pos = df[df['sentiment'] == 1]
df_neu = df[df['sentiment'] == 0]
df_neg = df[df['sentiment'] == -1]

# Upsample minority classes
df_neu_up = resample(df_neu, replace=True, n_samples=len(df_pos), random_state=42)
df_neg_up = resample(df_neg, replace=True, n_samples=len(df_pos), random_state=42)

# Combine and shuffle
df = pd.concat([df_pos, df_neu_up, df_neg_up])
df = df.sample(frac=1, random_state=42).reset_index(drop=True)

# Check new balance
print("Balanced class distribution:")
print(df['sentiment'].value_counts())

df['comment'].duplicated().sum()
print("Rows in data before removing duplicate rows ",df.shape[0])

df = df.drop_duplicates(subset=['comment'])

print("Rows in data after removing duplicate rows ",df.shape[0])
from sklearn.utils import resample

```

```

# Separate by sentiment
df_pos = df[df['sentiment'] == 1]
df_neu = df[df['sentiment'] == 0]
df_neg = df[df['sentiment'] == -1]

# Upsample neutral and negative to match positive count
df_neu_up = resample(df_neu, replace=True, n_samples=len(df_pos), random_state=42)
df_neg_up = resample(df_neg, replace=True, n_samples=len(df_pos), random_state=42)

# Combine all into a balanced dataset
df = pd.concat([df_pos, df_neu_up, df_neg_up])
df = df.sample(frac=1).reset_index(drop=True)

# Confirm new class distribution
print("Balanced class counts:")
print(df['sentiment'].value_counts())

(
    df['sentiment']
    .value_counts()
    .plot(kind='bar')
)

# remove whitespaces

df['comment'] = df['comment'].str.lstrip()

def count_words(text):
    return len(text.split(" "))

df['word_count'] = df['comment'].apply(count_words)

# statistics on word_count

```

```

df['word_count'].agg(['min','max','mean'])

# convert all the sentences to lowercase

df['comment'] = df['comment'].str.lower()

# convert all the sentences to lowercase

df['comment'] = df['comment'].str.lower()

def removing_numbers(text):
    text="".join([i for i in text if not i.isdigit()])
    return text

def removing_punctuations(text):
    ## Remove punctuations
    text = re.sub('[%s]' % re.escape("""!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~"""), ' ', text)
    text = text.replace(' ', '')
    return text

def removing_urls(text):
    url_pattern = re.compile(r'https?://\S+|www\.\S+')
    return url_pattern.sub(r'', text)

df['comment'].head()

def preprocess_text(text):
    text = removing_numbers(text)
    text = removing_urls(text)
    text = removing_punctuations(text)
    return text

df['comment'] = df['comment'].apply(preprocess_text)

# remove stopwords from comments

```

```

def remove_stopwords(text):
    stop_words = set(stopwords.words('english'))
    word_tokens = word_tokenize(text)
    filtered_sentence = [w for w in word_tokens if not w.lower() in stop_words]
    return " ".join(filtered_sentence)

df['comment'] = df['comment'].apply(remove_stopwords)

# lemmatize the text

# lemmatizer to be used instead. not working in kaggle.

def stemming(text):
    stemmer = PorterStemmer()
    text = [stemmer.stem(word) for word in text.split(" ")]
    return " ".join(text)

df['comment'] = df['comment'].apply(stemming)

# make X and y

X = df.drop(columns='sentiment')
y = df['sentiment']

X

# train test split the data

X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_state=True,stratify=y)

print("The shape of X_train is",X_train.shape)
print("The shape of X_test is",X_test.shape)

```



```

# plot the two graphs

fig = plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
y_train.value_counts().plot(kind='bar')
plt.title("Train Data")

plt.subplot(1,2,2)
y_test.value_counts().plot(kind='bar')
plt.title("Test Data")

# transform the output variable
le = LabelEncoder()

y_train = le.fit_transform(y_train)
y_test = le.transform(y_test)

# form the preprocessor

preprocessor = ColumnTransformer(transformers=[
    ('encode',CountVectorizer(decode_error='ignore'),'comment'),
    ('scale',StandardScaler(),['word_count'])
],n_jobs=-1)

preprocessor
preprocessor.fit_transform(X_train.sample(20)).toarray()
results = dict(search.cv_results_)
# This line ONLY affects file saving, not model output
pd.DataFrame(results).to_csv("results.csv", index=False)

import pickle as pkl

```

```

with open("results.pkl", 'wb') as res:
    pkl.dump(results, res)

# best model

best_model = search.best_estimator_
import os
import joblib

# Save in a local "models" folder
output_dir = 'models'
os.makedirs(output_dir, exist_ok=True)

joblib.dump(best_model, os.path.join(output_dir, "best_model.joblib"))

print("The train accuracy score is: ",accuracy_score(y_train,y_pred_train))
print("The train f1 score is: ",f1_score(y_train,y_pred_train,average='macro'))

print("#"*50)

print("The test accuracy score is: ",accuracy_score(y_test,y_pred_test))
print("The test f1 score is: ",f1_score(y_test,y_pred_test,average='macro'))
print(f"The avg cross val score is {score.mean():.4f}")
def preprocess_manual_comment(comment):
    # Clean and process the input just like training
    comment = comment.lower()
    comment = ".join([i for i in comment if not i.isdigit()])
    comment = re.sub('[%s]' % re.escape("!'#$%&'()*+,-./:;<=>?!?@[\\]^_`{|}~\""), ' ', comment)
    comment = comment.replace(':', '')
    comment = re.sub(r'https?://\S+|www\.\S+', '', comment)

```

```

# Remove stopwords

stop_words = set(stopwords.words('english'))

tokens = word_tokenize(comment)

filtered = [w for w in tokens if w not in stop_words]


# Stemming

stemmer = PorterStemmer()

stemmed = [stemmer.stem(word) for word in filtered]

processed_text = " ".join(stemmed)


return processed_text


def predict_sentiment(comment):

    processed_comment = preprocess_manual_comment(comment)

    word_count = len(processed_comment.split())


    # Wrap in DataFrame to match model input

    input_df = pd.DataFrame({'comment': [processed_comment], 'word_count': [word_count]})


    # Predict

    prediction = best_model.predict(input_df)[0]


    # Map back to label

    label = le.inverse_transform([prediction])[0]


    # Convert to numerical sentiment

    sentiment_map = {'Positive': 1, 'Negative': -1, 'Neutral': 0}

    return sentiment_map.get(label, "Unknown"), label


def predict_manual_comment_sentiment(comment):

```

```

processed = preprocess_manual_comment(comment)
wc = len(processed.split())

input_df = pd.DataFrame({'comment': [processed], 'word_count': [wc]})
pred = best_model.predict(input_df)[0] # This will be 0, 1, or 2

# Reverse lookup based on LabelEncoder order: [-1, 0, 1]
label_mapping = {
    0: "Negative", # encoded -1
    1: "Neutral", # encoded 0
    2: "Positive" # encoded 1
}

score_mapping = {
    0: -1,
    1: 0,
    2: 1
}

return label_mapping.get(pred, "Unknown"), score_mapping.get(pred, "Unknown")

print(predict_manual_comment_sentiment("This video is trash."))
print(predict_manual_comment_sentiment("Wow I loved every second of it!"))
print(predict_manual_comment_sentiment("It's just okay I guess."))

```

ENSEMBLE MODEL

(Combination of RandomForest, LR and SVR) –

```
!pip install transformers datasets

from google.colab import files

uploaded = files.upload()

# ✅ Disable WandB logs in Colab
os.environ["WANDB_DISABLED"] = "true"

# ✅ Load and clean dataset from Colab path
df = pd.read_csv("/content/Twitter_Data.csv.csv")
print("Available columns:", df.columns.tolist())
# Adjust column names below as per your dataset
# For example, use 'category' if it exists; else use 'sentiment' or 'label'
df = df[['text', 'sentiment']].dropna()
df.rename(columns={'text': 'clean_comment', 'sentiment': 'category'}, inplace=True)
df.rename(columns={'text': 'clean_comment'}, inplace=True)

# ✅ Train/test split for classic ML
X_train, X_test, y_train, y_test = train_test_split(df['clean_comment'], df['category'], test_size=0.2,
random_state=42)

vectorizer = TfidfVectorizer(max_features=6000, ngram_range=(1,2))
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

# ✅ Logistic Regression
logistic_model = LogisticRegression(C=5.0, max_iter=1000, solver='liblinear')
logistic_model.fit(X_train_vec, y_train)
y_pred = logistic_model.predict(X_test_vec)
print("\nLogistic Regression Performance:")
```

```
print(classification_report(y_test, y_pred))
```

```
# ✅ Random Forest Classifier
```

```
rf_model = RandomForestClassifier(n_estimators=300, max_depth=50, random_state=42)
```

```
rf_model.fit(X_train_vec, y_train)
```

```
y_pred_rf = rf_model.predict(X_test_vec)
```

```
print("\nRandom Forest Performance:")
```

```
print(classification_report(y_test, y_pred_rf))
```

```
# ✅ Inference using Logistic Regression
```

```
def predict_sentiment_logistic(text):
```

```
    vec = vectorizer.transform([text])
```

```
    return logistic_model.predict(vec)[0]
```

```
print("Predicted Sentiment (Logistic):", predict_sentiment_logistic("Buddhism is very peaceful and calming"))
```

```
# ✅ Label Mapping for RoBERTa
```

```
label_map = {-1: 0, 0: 1, 1: 2}
```

```
reverse_map = {0: -1, 1: 0, 2: 1}
```

```
df['label'] = df['category'].map(label_map)
```

```
# ✅ Convert to HuggingFace Dataset
```

```
hf_dataset = Dataset.from_pandas(df[['clean_comment', 'label']])
```

```
hf_dataset = hf_dataset.train_test_split(test_size=0.2)
```

```
# ✅ Tokenization for RoBERTa
```

```
tokenizer = RobertaTokenizer.from_pretrained("roberta-base")
```

```
def tokenize(example):
```

```
        return tokenizer(example["clean_comment"], truncation=True, padding="max_length",
max_length=128)
```

```
tokenized_dataset = hf_dataset.map(tokenize, batched=True)
```

```
#  Load RoBERTa model and set device
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
model = RobertaForSequenceClassification.from_pretrained("roberta-base", num_labels=3)
```

```
model.to(device)
```

```
#  Metrics for Trainer
```

```
def compute_metrics(eval_pred):
```

```
    logits, labels = eval_pred
```

```
    preds = np.argmax(logits, axis=1)
```

```
    precision, recall, f1, _ = precision_recall_fscore_support(labels, preds, average='weighted',
zero_division=0)
```

```
    acc = accuracy_score(labels, preds)
```

```
    return {"accuracy": acc, "f1": f1, "precision": precision, "recall": recall}
```

```
#  Training Arguments
```

```
training_args = TrainingArguments(
```

```
    output_dir="./roberta_results",
```

```
    per_device_train_batch_size=16,
```

```
    per_device_eval_batch_size=16,
```

```
    num_train_epochs=6,
```

```
    weight_decay=0.01,
```

```
    learning_rate=2e-5,
```

```
    warmup_steps=500,
```

```
    logging_dir="./logs"
```

```
)
```

 Trainer Setup

```
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=tokenized_dataset["train"],  
    eval_dataset=tokenized_dataset["test"],  
    compute_metrics=compute_metrics  
)
```

 Train the RoBERTa model

```
trainer.train()
```

 Evaluate the RoBERTa model

```
metrics = trainer.evaluate()  
  
print("📊 RoBERTa Evaluation Metrics:")  
  
roberta_results = {}  
  
for k, v in metrics.items():  
    if 'accuracy' in k or 'f1' in k or 'precision' in k or 'recall' in k:  
        print(f'{k.capitalize()}: {v:.4f}')  
        roberta_results[k] = v
```

 Summary comparison table

```
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
```

```
log_acc = accuracy_score(y_test, y_pred)
```

```
log_prec, log_rec, log_f1, _ = precision_recall_fscore_support(y_test, y_pred, average='weighted',  
zero_division=0)
```

```
rf_acc = accuracy_score(y_test, y_pred_rf)
```

```
rf_prec, rf_rec, rf_f1, _ = precision_recall_fscore_support(y_test, y_pred_rf, average='weighted',  
zero_division=0)
```



```

print(" 📄 Accuracy Comparison (Weighted Avg):")

print(f"Logistic Regression -> Accuracy: {log_acc:.4f}, Precision: {log_prec:.4f}, Recall: {log_rec:.4f}, F1: {log_f1:.4f}")

print(f"Random Forest -> Accuracy: {rf_acc:.4f}, Precision: {rf_prec:.4f}, Recall: {rf_rec:.4f}, F1: {rf_f1:.4f}")

print(f"RoBERTa -> Accuracy: {roberta_results['eval_accuracy']:.4f}, Precision: {roberta_results['eval_precision']:.4f}, Recall: {roberta_results['eval_recall']:.4f}, F1: {roberta_results['eval_f1']:.4f}")

```

✅ RoBERTa Inference Function

```

def predict_sentiment_roberta(text):
    model.eval()

    inputs = tokenizer(text, return_tensors="pt", truncation=True, padding=True, max_length=128)
    inputs = {k: v.to(device) for k, v in inputs.items()}

    with torch.no_grad():
        outputs = model(**inputs)

    predicted_class = torch.argmax(outputs.logits, dim=1).item()

    return reverse_map[predicted_class]

```

✅ Test RoBERTa Prediction

```

sample = "I love the peaceful teachings of Buddhism."

print("Predicted Sentiment (RoBERTa):", predict_sentiment_roberta(sample))

```

SVR Model –

```
from google.colab import files
uploaded = files.upload()
!ls
import io
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
from imblearn.over_sampling import RandomOverSampler
from imblearn.pipeline import Pipeline
from sklearn.svm import LinearSVC
df = pd.read_csv(io.StringIO(uploaded['Reddit_Data.csv'].decode('utf-8')))

# Display available columns and a data sample.
print("Columns in the dataset:", df.columns.tolist())
print("Dataset sample:")
print(df.head())

df['clean_comment'] = df['clean_comment'].fillna("")
# Also drop rows with missing labels if any.
df = df.dropna(subset=['category'])
X = df['clean_comment']
y = df['category']
print("\nLabel distribution after cleaning:")
print(y.value_counts())
X_train, X_test, y_train, y_test = train_test_split(
```

```

X, y, test_size=0.2, random_state=42, stratify=y
)
pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(ngram_range=(1,2), sublinear_tf=True, max_features=10000)),
    ('oversample', RandomOverSampler(random_state=42)),
    ('clf', LinearSVC(class_weight='balanced', random_state=42))
])
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Negative', 'Neutral', 'Positive'],
            yticklabels=['Negative', 'Neutral', 'Positive'])
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix")
plt.show()

# Step 9: Manual Prediction Section
def predict_manual_input(model_pipeline):
    """
    Repeatedly accepts manual text input and outputs the predicted label.
    Press ENTER without input to exit.
    """
    print("\nEnter manual text for prediction (press ENTER without input to exit):")
    while True:

```

```
user_input = input("Input text: ").strip()
if user_input == "":
    break
prediction = model_pipeline.predict([user_input])
print("Predicted label:", prediction[0])

# Run the manual prediction loop.
predict_manual_input(pipeline)
```

BERT Model –

```
!pip install transformers datasets torch scikit-learn pandas
```

```
!pip install transformers datasets -q
```

```
# ✅ STEP 2: Import libraries
```

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import classification_report
```

```
# ✅ STEP 3: Load and clean dataset
```

```
df = pd.read_csv("/content/Reddit_Data.csv") # Path to your file
```

```
df = df[['clean_comment', 'category']].dropna()
```

```
# ✅ STEP 4: Train/test split
```

```
X_train, X_test, y_train, y_test = train_test_split(df['clean_comment'], df['category'], test_size=0.2, random_state=42)
```

```
# ✅ STEP 5: TF-IDF Vectorization
```

```
vectorizer = TfidfVectorizer(max_features=5000)
```

```
X_train_vec = vectorizer.fit_transform(X_train)
```

```
X_test_vec = vectorizer.transform(X_test)
```

```
# ✅ STEP 6: Train Logistic Regression model
```

```
model = LogisticRegression()
```

```
model.fit(X_train_vec, y_train)
```

```
# ✅ STEP 7: Evaluation
```

```
y_pred = model.predict(X_test_vec)
```

```
print(classification_report(y_test, y_pred))
```

```
# ✅ STEP 8: Inference function
```

```
def predict_sentiment(text):
```

```
    vec = vectorizer.transform([text])
```

```
    return model.predict(vec)[0]
```

```
# Example:
```

```
print("Predicted Sentiment:", predict_sentiment("Buddhism is very peaceful and calming"))
```

```
# ✅ Step 2: Imports
```

```
import pandas as pd
```

```
import numpy as np
```

```
import torch
```

```
import os
```

```
from datasets import Dataset
```

```
from transformers import BertTokenizer, BertForSequenceClassification, TrainingArguments, Trainer
```

```
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
```

```
# ✅ Step 3: Disable WandB (optional, prevents login issues)
```

```
os.environ["WANDB_DISABLED"] = "true"
```

```
# ✅ Step 4: Load your dataset
```

```
df = pd.read_csv("/content/Reddit_Data.csv") # Use your CSV path
```

```
df = df[['clean_comment', 'category']].dropna()
```

```
# ✅ Step 5: Label mapping for 3-class classification
```

```
label_map = {-1: 0, 0: 1, 1: 2}
```

```
reverse_map = {0: -1, 1: 0, 2: 1}
```

```
df['label'] = df['category'].map(label_map)
```

```

# ✅ Step 6: Convert to Hugging Face Dataset
dataset = Dataset.from_pandas(df[['clean_comment', 'label']])
dataset = dataset.train_test_split(test_size=0.2)

# ✅ Step 7: Tokenizer
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

def tokenize(example):
    return tokenizer(example["clean_comment"], truncation=True, padding="max_length",
max_length=128)

tokenized_dataset = dataset.map(tokenize, batched=True)

# ✅ Step 8: Load BERT model with 3 output labels
model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=3)

# ✅ Step 9: Define evaluation metrics
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=1)
    precision, recall, f1, _ = precision_recall_fscore_support(labels, preds, average='weighted',
zero_division=0)
    acc = accuracy_score(labels, preds)
    return {"accuracy": acc, "f1": f1, "precision": precision, "recall": recall}

# ✅ Step 10: Training arguments
training_args = TrainingArguments(
    output_dir="./bert_results",
    evaluation_strategy="epoch",
    save_strategy="epoch",

```

```

    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=3,
    weight_decay=0.01,
    logging_dir="./logs",
    load_best_model_at_end=True,
    metric_for_best_model="f1"
)

```

 Step 11: Initialize Trainer

```

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset["train"],
    eval_dataset=tokenized_dataset["test"],
    tokenizer=tokenizer,
    compute_metrics=compute_metrics
)

```

 Step 12: Train the model

```

trainer.train()

```

 Step 13: Predict function

```

def predict_sentiment(text):

```

 Step 14: Test prediction

```

sample = "I love the peaceful teachings of Buddhism."

```

```

print("Predicted Sentiment:", predict_sentiment(sample)) # Should print -1, 0, or 1

```


APPENDIX B

CONFERENCE PRESENTATION

Conference List

My Conferences (1)

All Conferences

type to filter...

Name	Start Date	Location	External URL	Contact
4th 2025 IEEE World Conference on Applied Intelligence and Computing	7/26/2025	Delhi-NCR, India	https://scrs.in/conference/aic2025	Email Chairs

4th 2025 IEEE World Conference on Applied Intelligence and Computing : Submission (2095) has been created. Inbox x



Microsoft CMT <noreply@msr-cmt.org>
to me ▾

Sun 27 Apr, 23:58 (3 minutes ago)



Hello,

The following submission has been created.

Track Name: AIC2025

Paper ID: 2095

Paper Title: Sentimental analysis using social media comments

Abstract:

Social media has been a key source of public opinion, enabling researchers to tap into spontaneous and unmediated user-generated content. This paper investigates sentiment classification from Reddit comments to determine user attitudes in pre defined categories. The methodology starts with data cleaning and preprocessing text data followed by feature extraction using Term

Frequency-Inverse Document Frequency (TF-IDF), to handle class imbalance—a typical problem in real-world datasets—we employ oversampling using the RandomOverSampler algorithm. Preprocessed data is then used to train a Linear Support Vector Classifier (LinearSVC), which is selected based on its ability to cope with high-dimensional text features. Model assessment is done using accuracy, confusion matrices, and classification reports. Results show that the employment of TF-IDF combined with oversampling has a significant impact on enhancing model performance, particularly in skewed data environments. This paper presents a practical, scalable approach of performing sentiment analysis on social media data.

Created on: Sun, 27 Apr 2025 18:28:40 GMT

Last Modified: Sun, 27 Apr 2025 18:28:40 GMT

Authors:

- nsdinesh1318@gmail.com (Primary)
- as0229@srmist.edu.in
- muralin@srmist.edu.in

Primary Subject Area: Computational intelligence

Secondary Subject Areas:

Intelligent Systems (Hardware & Software)

APPENDIX C

PUBLICATION DETAILS

APPENDIX D

PLAGIARISM REPORT



Page 1 of 63 - Cover Page

Submission ID trn:oid::1:3230126283

Murali M

Dinesh project

ASSIGNMENT 11

Cloud 2023-24

SRM Institute of Science & Technology

Document Details

Submission ID

trn:oid::1:3230126283

Submission Date

Apr 27, 2025, 5:56 PM GMT+5:30

Download Date

Apr 27, 2025, 5:59 PM GMT+5:30

File Name

Report_Body_1_final.docx

File Size

2.0 MB

55 Pages

13,486 Words

85,771 Characters



Page 1 of 63 - Cover Page

Submission ID trn:oid::1:3230126283

9% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

- 118 Not Cited or Quoted 9%**
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 5% Internet sources
- 5% Publications
- 3% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.